CS-204: Object-Oriented Programming in Java

Personal Finance Tracker

A robust console-based application utilizing core OOP principles for effective budget management.

Problem Statement

Managing personal finances is increasingly complex for students and young professionals. Without a structured system, tracking daily expenses becomes chaotic, leading to poor saving habits and financial stress.

Manual tracking methods like spreadsheets or paper logs are highly error-prone, difficult to categorize, and lack immediate analytical insights required for proactive financial planning.

This project resolves these issues by providing an automated, strictly-typed Java application to seamlessly log, categorize, and analyze financial transactions.

Project Objectives

Automated Tracking

Provide a seamless interface to log daily income and expenses, ensuring accurate temporal data capture and balance calculations.

Smart Categorization

Group transactions by custom tags (e.g., 'Housing', 'Food', 'Leisure') to facilitate granular analysis of spending habits over time.

Financial Reporting

Generate detailed monthly summaries and visually represent data trends to empower users with actionable financial insights.

Core System Features

- **Secure User Authentication:** Encapsulated user profiles with credential validation to ensure data privacy across multiple users.
- **Dynamic Transaction Management:** CRUD (Create, Read, Update, Delete) capabilities for all financial entries using robust Java Collections.
- **Smart Budgeting & Alerts:** Set monthly thresholds per category; the system triggers warnings when expenses exceed 90% of the allocated budget.
- **Persistent Data Storage:** Utilization of Java File I/O (Serialization) to persist transaction state between application instances.
- **Robust Exception Handling:** Comprehensive try-catch blocks to handle `InputMismatchException` and custom `InsufficientBalanceException`.

OOP Implementation

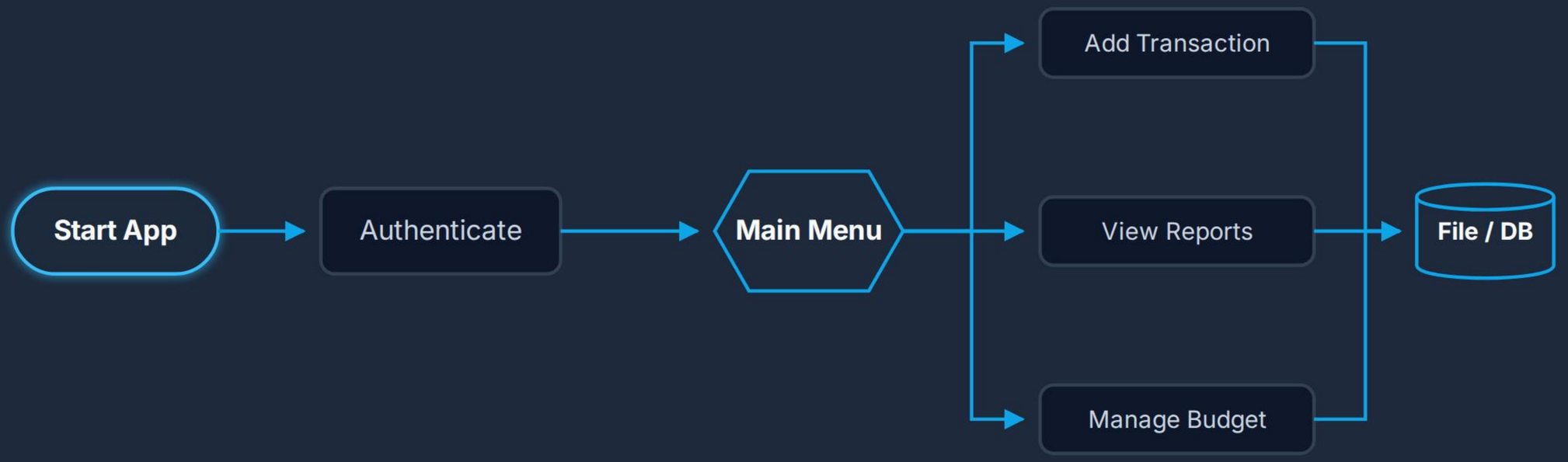
Inheritance & Polymorphism

The system utilizes a base abstract class `Transaction` which is inherited by specific `Income` and `Expense` subclasses. Polymorphism is achieved by overriding the `displayDetails()` method, allowing uniform iteration over a generic `List` while exhibiting subclass-specific behaviors.

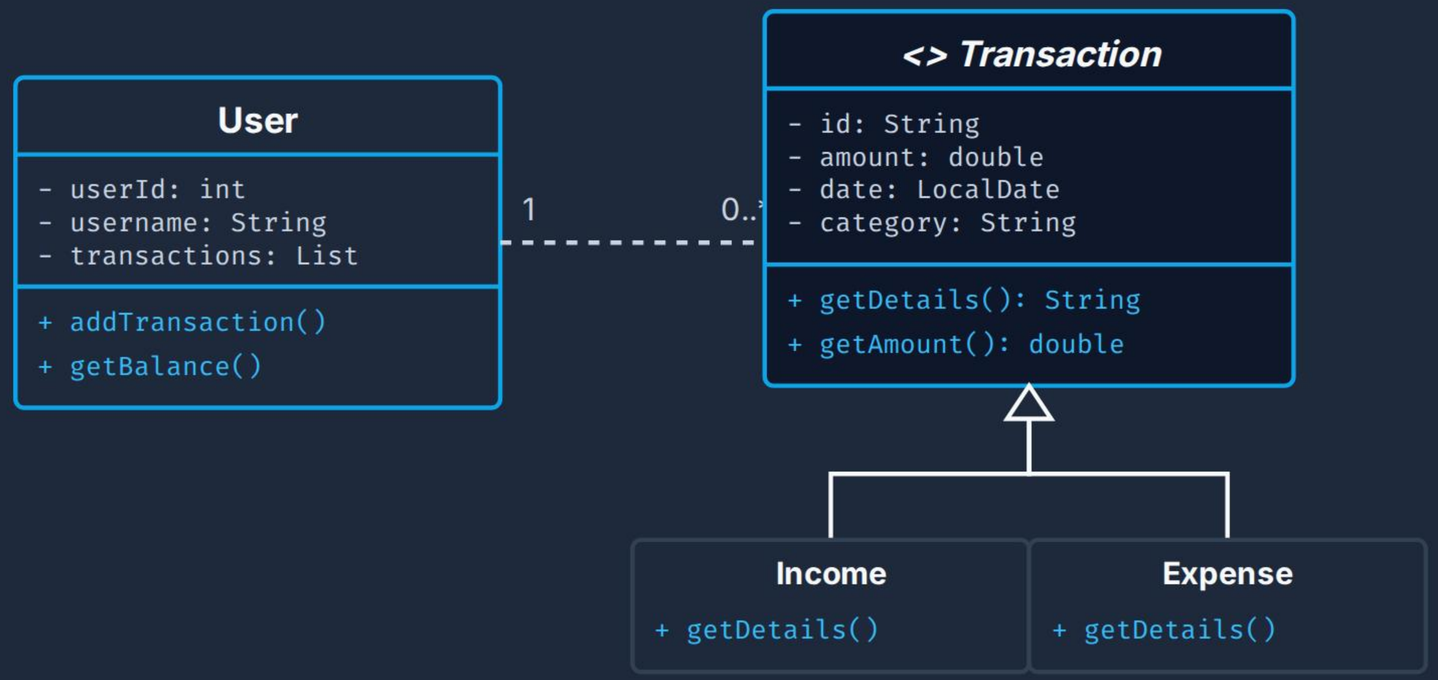
Encapsulation & Abstraction

All model state variables (balance, amount, dates) are declared `private` and accessed via strict getters/setters with validation logic. Complex subsystem interactions are hidden behind a `FinanceManager` interface, exposing only necessary operational methods to the UI layer.

System Execution Flow



UML Class Architecture



Technical Specifications

O(1)

Transaction Lookup Time

Optimized Data Structures

By utilizing Java's `HashMap` for mapping categories to transactions, we achieve constant time complexity for analytical queries, ensuring performance scaling even with years of logged data.

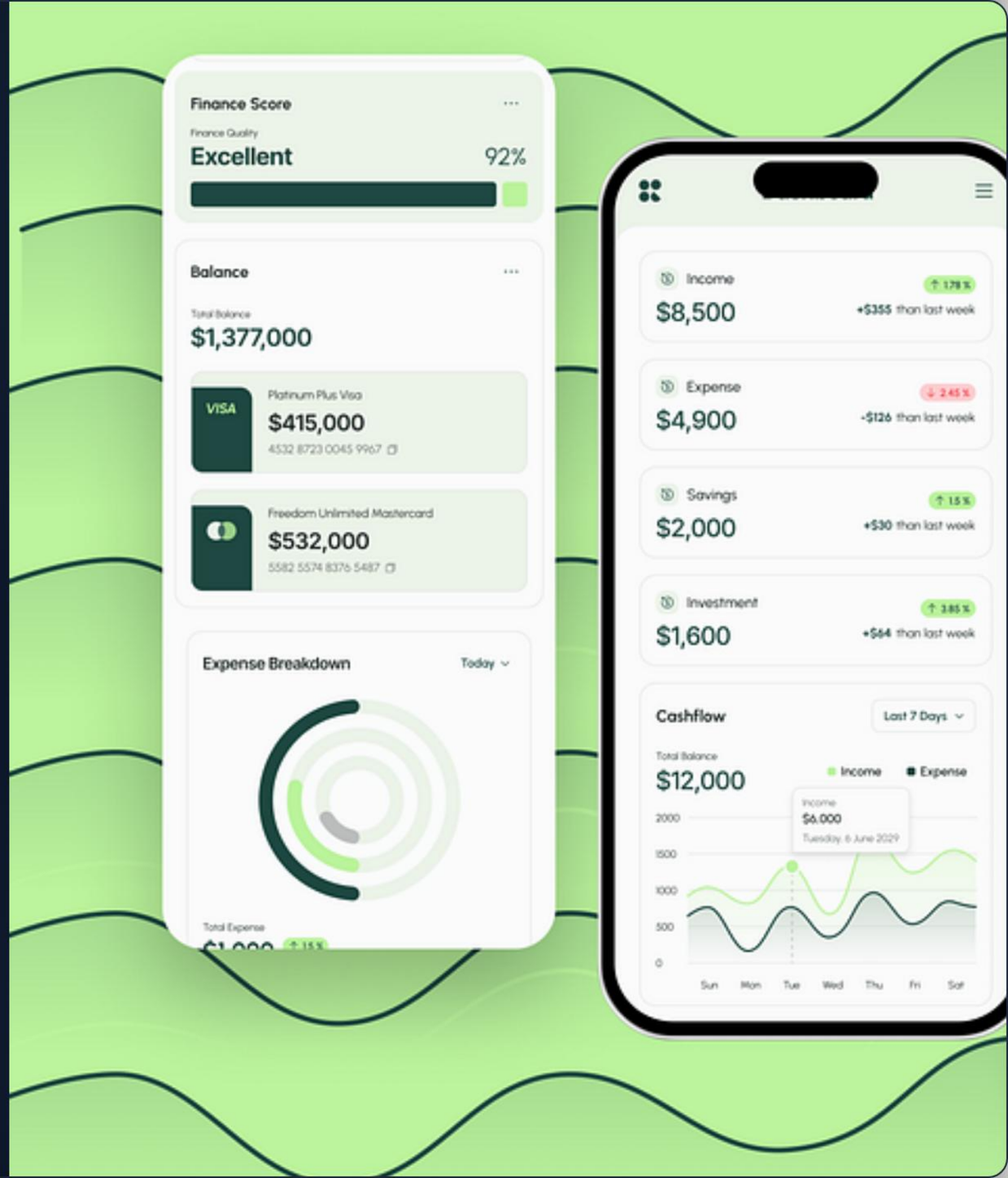
$$\text{Balance} = \sum_{i=1}^{n} \text{Income}_i - \sum_{j=1}^{m} \text{Expense}_j$$

User Interface Outlook

Intuitive & Error-Free

Whether implemented via a robust Command Line Interface (CLI) or transitioned to a JavaFX GUI, the system prioritizes user experience.

Input streams are heavily validated against incorrect data types. Navigation is state-driven, meaning the user can easily traverse back to main menus without sequence breaks.



Simulated Expense Analytics

- Housing & Utilities (40%)
- Food & Groceries (30%)
- Transportation (15%)
- Entertainment (15%)

Data is aggregated by overriding the `toString()` methods and parsing categorizations through standard Java Streams API.

Project Development Lifecycle

Phase 1: Design

Requirement gathering, establishing UML diagrams, and defining class hierarchies.

Phase 2: Core Logic

Implementing inheritance, encapsulation, and building the in-memory collections.

Phase 3: Persistence

Integrating Java File I/O for saving state, alongside custom Exception Handling.

Phase 4: Validation

Unit testing core methods, UI boundary checks, and final project packaging.

Thank You

Ready to discuss the codebase, architectural decisions, and OOP implementations.

```
Project Evaluated For: 4th Semester Java Lab
```

Image Sources

https://img.freepik.com/premium-photo/focused-businessman-business-person-accountant-office-worker-writing-ideas-notebook-paperwork-arabian-indian-student-write-notes-data-information-elearning-with-laptop-computer-educational-course_717737-25348.jpg

Source: www.freepik.com

https://cdn.dribbble.com/userupload/44421599/file/50fa92e7817f6fe84e69b26cd63f1a5d.png?resize=752x&vertical=center

Source: dribbble.com